OS orange Problem

SCREENSHOT 1A

```
venom@venom-QEMU-Virtual-Machine:~/PES1UG24CS241-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

SCREENSHOT 1B

```
venom@venom-QEMU-Virtual-Machine:~/PES1UG24CS241-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

SCREENSHOT 2A

```
venom@venom-QEMU-Virtual-Machine:~/PES1UG24CS241-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

SCREENSHOT 2B

```
venom@venom-QEMU-Virtual-Machine:~/PES1UG24CS241-pes-vcs$ xxd .pes/objects/5b/0410c33d2229bbe377a506f669680429c80cb9fa70859a576f8463633e27cf
00000000: 7472 6565 2039 3000 3130 3036 3434 2061  tree 90.100644 a
00000010: 2e74 7874 002c f8d8 3d9e e295 43b3 4a87  .txt.,..=...C.J.
00000020: 7274 21fd ecb7 e3f3 a183 d337 6390 25de  rt!........7c.%.
00000030: 576d b9eb b431 3030 3634 3420 622e 7478  Wm...100644 b.tx
00000040: 7400 e00c 50e1 6a2d f38f 8d6b f809 e181  t...P.j-...k....
00000050: ad02 48da 6e67 19f3 5f9f 7e65 d6f6 0619  ..H.ng.._.~e....
00000060: 9f7f                                     ..
```

SCREENSHOT 3A

```
venom@venom-QEMU-Virtual-Machine:~/PES1UG24CS241-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt
```

SCREENSHOT 3B

```
venom@venom-QEMU-Virtual-Machine:~/PES1UG24CS241-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f file2.txt
```

SCREENSHOT 4A

```
./pes log
Initialized empty PES repository in .pes/
Committed: fa21c828c188... Initial commit
Committed: 8a1e5224b81e... Add world
Committed: 1159a142d51e... Add farewell
commit 1159a142d51e96f98e0b93924bc43f5cfe6d4bfa5ebbea5e4bde8af1669a34ca
Author: Kumar Mahit <PES1UG24CS241>
Date:   1776798905

    Add farewell

commit 8a1e5224b81e2f4ade8832835ad2c49864e4899e082a5c78083eed37a285dcb4
Author: Kumar Mahit <PES1UG24CS241>
Date:   1776798905

    Add world
◆

commit fa21c828c1882c0dd7dec3defcf649525da06d53abc2c4d5925867ba4300f605
Author: Kumar Mahit <PES1UG24CS241>
Date:   1776798905

    Initial commit
```

SCREENSHOT 4B

```
venom@venom-QEMU-Virtual-Machine:~/PES1UG24CS241-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/11/59a142d51e96f98e0b93924bc43f5cfe6d4bfa5ebbea5e4bde8af1669a34ca
.pes/objects/33/5cc937de1483494a47a88a48c37be968c1af5e65add1ab572cc8419e9b9267
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/8a/1e5224b81e2f4ade8832835ad2c49864e4899e082a5c78083eed37a285dcb4
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/c1/3c9de65743a09cccaab64a44c7cf6ed462d9bd5efad5bb8c93865027b1edf6
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/fa/21c828c1882c0dd7dec3defcf649525da06d53abc2c4d5925867ba4300f605
.pes/refs/heads/main
```

SCREENSHOT 4C

```
venom@venom-QEMU-Virtual-Machine:~/PES1UG24CS241-pes-vcs$ cat .pes/refs/heads/main
1159a142d51e96f98e0b93924bc43f5cfe6d4bfa5ebbea5e4bde8af1669a34ca
venom@venom-QEMU-Virtual-Machine:~/PES1UG24CS241-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Q5.1: A branch in Git is just a file in `.git/refs/heads/` containing a commit hash. Creating a branch is creating a file. Given this, how would you implement `pes checkout <branch>` — what files need to change in `.pes/`, and what must happen to the working directory? What makes this operation complex?

Answer:
To implement **`pes checkout <branch>`**, the system must first update the `.pes/HEAD` file so that it points to the selected branch (e.g., `ref: refs/heads/<branch>`). It then reads the commit hash stored in `.pes/refs/heads/<branch>` and loads the corresponding commit object and its associated tree.

The working directory must be updated to match the tree of the target commit. This involves deleting files that do not exist in the target branch, creating new files that exist in the target tree, and overwriting files that differ.

This operation is complex because it must handle uncommitted changes safely. Overwriting modified files can lead to data loss, so the system must detect conflicts and prevent checkout when necessary. Additionally, reconstructing directory structures from tree objects adds further complexity.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

Answer:
To detect a dirty working directory, the system compares the current working directory with the index and the target branch. For each tracked file, the system computes the

hash of the current file contents and compares it with the hash stored in the index. If the hashes differ, the file has been modified.

Next, the system compares the same file with the version in the target branch's tree. If the file is both locally modified and different in the target branch, this creates a conflict.

In such cases, checkout must be refused to prevent overwriting the user's uncommitted changes. This ensures data safety by detecting modifications using only hashes stored in the index and object store.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

Answer:
In a detached HEAD state, HEAD points directly to a commit hash rather than a branch. When new commits are made in this state, they are created normally but are not associated with any branch. As a result, these commits are not referenced and can become unreachable over time.

A user can recover these commits by using the commit hash shown in the log. They can create a new branch pointing to that commit, which preserves it permanently. Without creating a branch, these commits may eventually be removed by garbage collection.

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Answer:
The algorithm begins by identifying all branch heads stored in `.pes/refs/heads/`. From each branch head, the system traverses the commit graph recursively. For every commit, it visits the associated tree and all referenced blobs.

During traversal, each visited object hash is stored in a hash set, marking it as reachable. After traversal is complete, the system scans the entire object store and deletes any object whose hash is not present in the reachable set.

A hash set is used because it provides constant-time lookup and prevents duplicate processing.

For a repository with 100,000 commits, each commit typically references one tree and multiple blobs. This results in approximately 100,000 commit objects, 100,000 tree objects, and several hundred thousand blob objects. Therefore, the system may need to visit around 600,000 to 800,000 objects in total.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Answer:
Running garbage collection concurrently with a commit operation is dangerous because it can lead to a race condition. During a commit, new objects such as blobs, trees, and commits are created but may not yet be fully linked into the reference structure.

If garbage collection runs at the same time, it may identify these newly created objects as unreachable and delete them before the commit completes. This results in a corrupted repository where a commit references missing objects.

Git avoids this problem by using locking mechanisms to prevent concurrent modifications, maintaining reference logs (reflogs) to keep recently created objects safe, and applying a grace period before deleting unreachable objects. These safeguards ensure that objects involved in active operations are not mistakenly removed.